Multimodal Sentiment Analysis via Multi-Layer Feature Fusion and Multi-Task Learning

Manas Inamdar 2023102052
Soham Jahagirdar 2023102046
Parth Tokekar 2023102041
Gautam Gandhi 2023102059

Abstract

Multimodal sentiment analysis (MSA) tries to figure out how positive or negative someone feels by looking at what they say, how they say it, and what their face shows. This report describes where we are in our project of re-implementing the UFEN+MTFN model from Cai et al. (2025). The model has two parts: UFEN, which learns good features for each modality on its own, and MTFN, which combines all three modalities using cross-modal attention and a Transformer encoder-decoder. We trained our model on the CMU-MOSI dataset and currently achieve a binary accuracy (neg/pos) of 77.7% and Pearson correlation of 0.618 with 442K parameters. We explain what we built, what results we have, why our numbers are below the paper's target, and what we plan to do next to close that gap.

1 Introduction

When people talk, they send information through several channels at the same time. Their words carry meaning. Their voice carries emotion through tone and pace. Their face shows reactions through expressions. Multimodal Sentiment Analysis (MSA) is the task of automatically deciding whether a person is being positive or negative by looking at all these channels together, rather than reading the words alone.

This matters in many real situations. Think of a customer review video online. The person might say "it was fine" but their voice sounds flat and their face looks disappointed. A system that only reads the words will miss the real feeling. By combining text, audio, and video, we can build systems that understand people much more accurately.

1.1 Why Is This Problem Hard?

Making three very different types of data work together is not simple. There are three main problems.

The data looks completely different across modalities. Text is a sequence of 768-number word embeddings from BERT. Audio is a sequence of 74 acoustic measurements per time step. Video is a sequence of 35 face movement measurements per time step. These numbers live in totally different spaces and cannot be compared directly. A model needs to first put them all into a common space before they can be combined.

The modalities often say opposite things. This is especially common with sarcasm. Someone might say "I absolutely loved it" while their voice is flat and their face is blank or even frowning. If we just average the signals from each modality, the conflicting ones cancel each other out and we get the wrong answer. A good model needs to decide which modality to trust more in each case.

Text is much more useful than audio or video for sentiment. In the CMU-MOSI dataset, if you only use text features you already get around 73% accuracy. Audio and video add only a few extra points. But those few points matter, and the cases where audio and video help are often exactly the hard cases (like sarcasm). So a model cannot just ignore audio and video even though they are weaker signals.

1.2 Our Approach

The paper we are re-implementing, Cai et al. (2025), handles all three problems with a two-stage design. In Stage 1, a network called UFEN processes each modality by itself. It uses a combination of recurrent layers, convolution, and self-attention to build a good, compact representation for each modality independently. In Stage 2, a network called MTFN takes these three representations and applies cross-modal attention across all six possible directed pairs. This lets each modality "look into" the others to find information it is missing. Finally, a Transformer encoder-decoder block refines the combined result.

The training also uses five separate losses, one for each of the three modalities alone and two for the combined result. This forces the model to keep all three modalities useful throughout training, rather than learning to ignore the weaker ones.

1.3 Our Contributions

We are implementing this model from scratch in PyTorch. The specific things we have done so far are:

- A full PyTorch implementation of both UFEN and

MTFN, with every design choice documented, including the parts the original paper did not fully describe.

- A complete training and evaluation pipeline for CMU-MOSI, including proper data loading, Z-score normalisation with training-set statistics, gradient clipping, early stopping, and all five standard metrics.
- A working model with 442,501 parameters that achieves Acc-2 (neg/pos) = 77.7% and Pearson correlation = 0.618, giving us a solid starting baseline.
- An ablation study confirming which parts of the model matter most, plus a clear explanation of why our numbers are lower than the paper's target.

2 Related Work

2.1 Representing Each Modality

Early MSA systems used hand-designed features. For text, people counted words or used n-gram vectors. For audio, they measured average pitch or energy. For video, they tracked which facial muscles were moving. These features were easy to build but not very powerful.

Deep learning made a big difference. LSTM and GRU networks could read a sequence of features and learn what patterns meant positive or negative sentiment (Poria et al., 2017). The biggest jump in text quality came from BERT (Devlin et al., 2019), which reads an entire sentence and gives each word a vector that depends on its context. For example, the word "sick" has a completely different meaning in "that performance was sick" (slang for great) versus "I felt sick" (ill). BERT handles this naturally. COVAREP (Degottex et al., 2014) provides a standard set of voice quality measurements for audio, and FACET 4.2 provides facial action unit measurements for video. These three feature extractors together are now the standard starting point for CMU-MOSI experiments.

2.2 Combining Modalities (Fusion)

There are three general families of fusion strategies: early, late, and attention-based fusion.

Early fusion combines features before training. The simplest version concatenates all feature vectors into one big vector and feeds it into a neural network (Zadeh et al., 2016). This is fast but ignores the fact that text, audio, and video have very different structures.

Tensor Fusion (TFN) (Zadeh et al., 2017) takes early fusion further. It computes the outer product of all three modality vectors, which creates a tensor that explicitly represents every combination of features from all three modalities at once. The problem is that this tensor is huge: if each modality vector has d dimensions, the tensor has d^3 elements. Low-rank Multimodal Fusion (LMF) (Liu et al., 2018) fixes this by factorising the weight tensors into smaller pieces, which gives similar accuracy at much lower computation cost.

Late fusion trains a separate model for each modality and combines their predictions at the end. This is simple but cannot model interactions between modalities at the feature level.

Attention-based fusion is the most powerful modern approach. The key idea is that instead of just stacking or averaging features, we let each modality gather relevant information from the others using attention. The Multimodal Transformer (MulT) (Tsai et al., 2019) applies cross-modal attention so the text can attend to audio time steps and vice versa. This works well even when the modalities are not aligned in time. Memory Fusion Network (MFN) (Zadeh et al., 2018a) stores multi-view interactions in an explicit memory and updates it at every step. Graph-MFN (Zadeh et al., 2018b) extends this into a graph structure. MISA (Hazarika et al., 2020) separates each modality's representation into two parts: one that captures what all modalities share, and one that captures what is unique to that modality.

2.3 Multi-Task Learning for Sentiment

A common problem in MSA training is that the model learns to mostly rely on text, since text is so much more informative than audio or video. Multi-task learning is a way to fight this.

The idea is to make the model predict the sentiment score not just from the combined features, but also from each modality alone. This forces the model to make each modality representation useful by itself, which in turn makes the combined result better.

Self-MM (Yu et al., 2021) is a strong example of this approach. It automatically generates separate sentiment labels for each modality using a self-supervised algorithm, then uses these as targets for per-modality heads. This achieves much better results than training with only one combined output. MMIM (Han et al., 2021) is similar in spirit but uses information-theoretic objectives: it maximises the mutual information between each modality's features and the final prediction.

The model we are re-implementing (Cai et al., 2025) combines cross-modal attention fusion with multi-task learning. It builds on the ideas of both MulT and Self-MM, and adds a Transformer encoder-decoder on top for an extra refinement step. Our job is to re-implement and understand all of these pieces.

3 Dataset

3.1 CMU-MOSI Overview

We use the **CMU-MOSI** dataset (Zadeh et al., 2016) for all experiments. CMU-MOSI contains 2,199 short video clips taken from YouTube movie review videos. Each clip is one sentence (called an utterance). A human gave each clip a sentiment score between -3 (very negative) and $+3$ (very positive). A score of $+1$ or $+2$ is mildly positive, -1 or -2 is mildly negative, $+3$ or -3 are at the extremes.

CMU-MOSI is the most common benchmark for MSA. Almost every paper in this area reports results on it, which makes it easy to compare methods. It does come with some challenges though. The dataset is small (only 1,283 training samples), which means models can easily overfit. The label distribution is also slightly unbalanced, with more negative samples than positive ones. These properties make it a good but difficult testbed.

3.2 Modality Features

We use the standard pre-extracted, word-aligned features. We do not extract features ourselves. The three modality feature sets are described below.

Text ($d_t = 768$). Each word in the sentence is a 768-dimensional vector from BERT-base-uncased (Devlin et al., 2019). BERT reads the full sentence simultaneously and gives each word a representation that reflects its meaning given all the other words around it. We use BERT as a fixed feature extractor at this stage (no fine-tuning).

Audio ($d_a = 74$). Each time step of the audio is described by 74 numbers from the COVAREP toolkit (Degottex et al., 2014). These 74 numbers measure voice properties like pitch (how high or low the voice is), formants (the resonance of the vocal tract), and glottal source parameters (properties of how the vocal cords vibrate). Together they give a compact description of the speaker's voice quality at each moment.

Visual ($d_v = 35$). Each video frame is described by 35 numbers from the FACET 4.2 toolkit. These cover facial action units (which face muscles are active), eye openness, and geometric measurements like the distance between the eyes. The visual features are the weakest predictor of sentiment on their own but can help in cases where the face clearly contradicts the words.

3.3 Preprocessing Steps

All sequences are padded or cut to exactly $T = 30$ time steps. This gives every sample the same shape, which is required for batched training.

The audio and visual features have a known quality problem: some frames contain extremely large negative values (around -3.4×10^{38}) because COVAREP fills silent frames with these placeholder values. If we do not remove them, the Z-score normalisation step will produce useless statistics. Our preprocessing pipeline handles this in order:

1. Clip all values to the range $[-10^4, 10^4]$ to remove the extreme placeholders.
2. Replace any remaining NaN or infinity values with zero.
3. Compute mean and standard deviation from the training set only, then apply these to all three splits (train, val, test).

We do not normalise the BERT text features. BERT produces values in a reasonable range already, and changing them could harm the representations.

3.4 Data Splits

The train/val/test splits are fixed for CMU-MOSI and are the same across all published papers (Table 1).

Split	Samples	Label range	Modalities
Train	1,283	$[-3, +3]$	Text / Audio / Visual
Validation	214	$[-3, +3]$	Text / Audio / Visual
Test	686	$[-3, +3]$	Text / Audio / Visual

Table 1: CMU-MOSI dataset splits. All modalities are available for every sample.

4 Methodology

4.1 Task Definition

The input is one video clip described by three sequences: text $X_t \in \mathbb{R}^{T \times 768}$, audio $X_a \in \mathbb{R}^{T \times 74}$, and visual $X_v \in \mathbb{R}^{T \times 35}$, all padded to $T = 30$ steps. The output is a single number predicting sentiment on the $[-3, +3]$ scale.

During training the model produces five outputs: three unimodal predictions (Y_t, Y_a, Y_v), a combined prediction from the cross-modal attention block (Y_m), and a final refined prediction from the encoder-decoder ($Y_{m'}$). At test time only $Y_{m'}$ is used.

4.2 Overall Architecture

The pipeline has two stages, shown in Figure 1.

Stage 1 (UFEN) runs the same multi-step feature extraction pipeline on each modality separately. Three independent UFEN networks (one per modality) each output a feature sequence of shape $T \times d_m$.

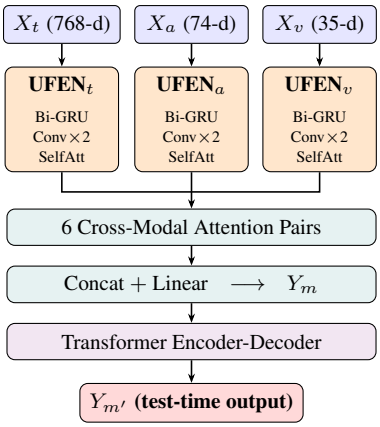

Figure 1: The UFEN+MTFN architecture. Each UFEN independently processes one modality (Bi-GRU, Conv×2, SelfAtt) and also produces a unimodal prediction (Y_t, Y_a, Y_v) used only during training. MTFN fuses all three representations via six directed cross-modal attention pairs, then refines the result through a Transformer encoder-decoder to give $Y_{m'}$, the final test-time output.

Stage 2 (MTFN) takes these three sequences and combines them using six directed cross-modal attention operations. The results are concatenated, projected to a first prediction Y_m, then passed through a Transformer encoder-decoder to get the final prediction $Y_{m'}$.

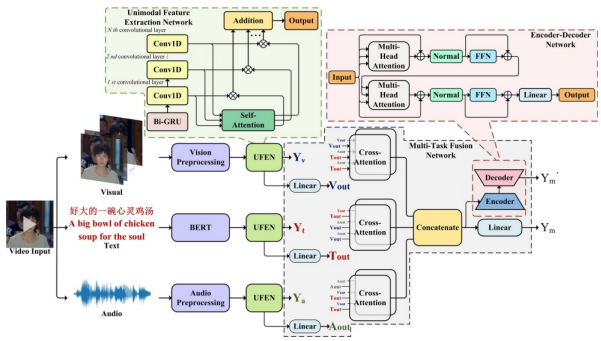

Figure 2: The overall architecture of our method. The Ym, Yt, Ya, Yv, Ym' are the prediction outputs of the multi-modal task, the three single-modal tasks and the fusion feature reconstruction task respectively. Ym' is used as the multimodal sentiment output.

4.3 Stage 1: UFEN in Detail

UFEN has five steps. All three modality networks share the same structure but have their own separate weights.

Step 1 – Bi-GRU.. The raw input $X_i \in \mathbb{R}^{T \times D_i}$ is read by a bidirectional GRU:

$$X_i^{\text{BiGRU}} = \text{BiGRU}(X_i) \in \mathbb{R}^{T \times d_m} \tag{1}$$

where $d_m = 2h$, $h = 32$ per direction so $d_m = 64$. Reading the sequence in both directions helps because sentiment often depends on context that comes later. For example, "not bad at all" is positive, but if we only read left to right the word "bad" might be misleading.

Step 2 – Two Conv1D layers.. We apply $l = 2$ stacked 1D convolutional layers:

$$X_i^{\text{layer}} = \text{Conv1D}^{\text{layer}}\big(X_i^{\text{BiGRU}},\ K{=}3,\ F{=}64\big) \tag{2}$$

Each convolution looks at 3 consecutive time steps at once and learns what local patterns are important. The first layer detects simple local features; the second builds on these to detect slightly wider patterns. Same-padding keeps the output length at T.

Step 3 – Self-Attention Gate.. After each convolution, we apply a self-attention mechanism:

$$\text{Attention}(X) = \text{softmax}\left(\frac{XX^T}{\sqrt{d_k}}\right) X \tag{3}$$

This lets each time step look at all other time steps and decide how much attention to give them. The output is used as a multiplicative gate on the convolution output:

$$X_i^{\text{layerAtt}} = \text{Attention}(X_i^{\text{layer}}) \tag{4}$$

$$X_i^{\text{self-att}} = X_i^{\text{layer}} \otimes X_i^{\text{layerAtt}} \tag{5}$$

where $\otimes$ is element-wise multiplication. Time steps that are more important for sentiment get amplified; less relevant ones get scaled down. This is different from regular self-attention because the output is used as a *gate*, not added back.

Step 4 – Sum across layers.. Each gated output is projected back to size d_m and all layers are added:

$$X_{\text{layer}} = \text{Linear}(X_i^{\text{self-att}}) \in \mathbb{R}^{T \times d_m} \tag{6}$$

$$X_i^{\text{fusion}} = X_1 \oplus X_2 \oplus \cdots \oplus X_l \tag{7}$$

where $\oplus$ is element-wise addition. Adding layer outputs (instead of using only the last one) means the final representation keeps both the local patterns from layer 1 and the broader patterns from layer 2.

Step 5 – Unimodal prediction head.. We average across time and apply a linear layer:

$$Y_i = W_i \cdot \text{MeanPool}(X_i^{\text{fusion}}) + b_i \tag{8}$$

This prediction is used only during training as part of the multi-task loss. It forces each UFEN to learn a meaningful representation that can predict sentiment on its own.

4.4 Stage 2: MTFN in Detail

Cross-modal attention.. The central idea of MTFN is that each modality can help fill in gaps in the others. The audio signal might confirm that a sentence is sarcastic; the visual signal might confirm that a neutral-sounding sentence is actually angry. Cross-modal attention allows modality i to query the other modality j and pull out the information it needs.

For each ordered pair (i, j), we project both sequences to a shared dimension $D_e = 64$ and compute multi-head attention with modality i as the query and modality j as key/value:

$$Q_i = W_{Q_i} W_i X_i, \quad K_j = W_{K_j} W_j X_j$$
$$V_j = W_{V_j} W_j X_j \tag{9}$$
$$\text{CA}_{i \to j} = \text{LN}(\text{MHAtt}(Q_i, K_j, V_j)) \tag{10}$$

The projection matrices W_i per modality are shared across all pairs involving that modality. This means text always maps to the same embedding space regardless of whether it is acting as query or being queried. All six directed pairs are computed: text→audio, text→visual, audio→text, audio→visual, visual→text, visual→audio. Each gives a tensor of shape (B, T, D_e).

Concatenation and first multimodal prediction.. The six attention outputs are averaged over the time dimension and joined together:

$$\text{Fused} = \text{Concat}[\text{MeanPool}(\text{CA}_{i \to j})]_{(i,j)} \in \mathbb{R}^{6D_e} \tag{11}$$

A linear layer maps this to the first multimodal prediction:

$$Y_m = W_m \cdot \text{Fused} + b_m \tag{12}$$

Transformer encoder-decoder.. The fused vector has 6 chunks of size D_e, so we reshape it to $(B, 6, D_e)$ and treat it as a 6-element sequence. One Transformer encoder block refines this sequence:

$$Z_{\text{enc}} = \text{LN}\big(\text{Input} + \text{MHAtt}(\text{Input})\big) \tag{13}$$
$$\text{Out}_{\text{enc}} = \text{LN}\big(Z_{\text{enc}} + \text{FFN}(Z_{\text{enc}})\big) \tag{14}$$

One Transformer decoder block then uses Out_{enc} as context:

$$Z_{\text{dec}} = \text{LN}\big(\text{Input} + \text{MHAtt}(\text{Input}, \text{Out}_{\text{enc}})\big) \tag{15}$$
$$\text{Out}_{\text{dec}} = \text{LN}\big(Z_{\text{dec}} + \text{FFN}(Z_{\text{dec}})\big) \tag{16}$$

Averaging over the 6 positions and projecting gives the final prediction:

$$Y_{m'} = W_{de} \cdot \text{MeanPool}(\text{Out}_{\text{dec}}) + b_{de} \tag{17}$$

$Y_{m'}$ is the output used at test time.

Hyperparameter	Value	Source
Learning rate	5×10^{-3}	Paper (Table 2)
Batch size	32	Paper (Table 2)
Max epochs	30	Paper (Table 2)
Early stop patience	8	Paper (Table 2)
Optimizer	Adam	Paper
Loss function	MSE	Paper
Self-Att heads	1	Paper (Table 2)
Cross-Att heads	4	Paper (Table 2)
Att dropout	0.2	Paper (Table 2)
Dropout	0.1	Paper (Table 2)
Conv layers (l)	2	Paper (ablation)
d_m (GRU hidden $\times 2$)	64	Assumed
D_e (cross-att dim)	64	Assumed ($= d_m$)
Conv kernel size K	3	Assumed
Conv filters F	64	Assumed ($= d_m$)
Enc/Dec heads	4	Assumed
FFN dim	256	Assumed ($4 \times d_m$)
Trainable parameters	442,501	Measured

Table 2: All hyperparameters used. "Assumed" values are not given in the paper.

4.5 Multi-Task Loss

All five predictions are trained simultaneously. The total loss is the sum of five MSE losses:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_t + \mathcal{L}_a + \mathcal{L}_v + \mathcal{L}_f + \mathcal{L}_{\text{ed}} \tag{18}$$

where each term is:

$$\mathcal{L}_x = \frac{1}{N} \sum_{n=1}^{N} (Y^n - \hat{Y}_x^n)^2 \tag{19}$$

All five use the same ground-truth label. The three unimodal losses are the key to making multi-task learning work. Without them the model can ignore audio and visual completely, since text alone already gives a decent accuracy. The unimodal losses prevent this by directly penalising poor unimodal predictions.

4.6 Design Decisions

The original paper leaves several hyperparameters unspecified. Table 2 shows every setting we use, labelled as either coming from the paper or chosen by us. The most important missing values are the GRU hidden size, the encoder-decoder hidden size, and the conv filter size. We chose $d_m = D_e = F = 64$ as a reasonable starting point for a small dataset.

5 Experimental Setup

5.1 Implementation

Our full implementation is publicly available at: https://github.com/Gautam-Gandhi/

multimodal-sentiment-analysis

We wrote the full model in PyTorch 2.0. All experiments run on an NVIDIA RTX 3050 Laptop GPU. Each training run finishes in about 5–8 minutes. We split the code into separate files for dataset loading (`dataset.py`), each model component (`ufen.py`, `mtfn.py`, `msa_model.py`), metrics (`metrics.py`), and training (`train.py`). A JSON config file controls all hyperparameters so we can run experiments without changing any code.

We use the Adam optimizer with learning rate 5×10^{-3}. We also tried applying gradient clipping at norm 1.0. The paper does not mention this, but without it the training occasionally diverges in the first few epochs due to very large gradient values. Clipping fixed this completely.

5.2 Evaluation Metrics

We compute six metrics on the test set, following the CMU-MOSI standard protocol (Yu et al., 2021; Hazarika et al., 2020):

- **Acc-2 (nn/np)**: Binary accuracy deciding positive vs. negative. "nn" uses all samples with threshold 0. "np" uses only non-zero labelled samples, threshold 0. The "np" version is the main metric most papers report.
- **F1 (nn/np)**: F1-score for the same two binary classifications.
- **MAE**: Mean absolute error on the continuous $[-3, +3]$ scale. Lower is better.
- **Corr**: Pearson correlation between our predictions and the true labels.
- **Acc-7**: Accuracy when both predictions and labels are rounded to the nearest integer (7 possible values from -3 to $+3$).

Higher is better for all metrics except MAE.

5.3 Baselines

We compare against results from the original paper for: **EF-LSTM** (Zadeh et al., 2016), **LF-DNN**, **TFN** (Zadeh et al., 2017), **LMF** (Liu et al., 2018), **MFN** (Zadeh et al., 2018a), **Graph-MFN** (Zadeh et al., 2018b), **MulT** (Tsai et al., 2019), **MISA** (Hazarika et al., 2020), **Self-MM** (Yu et al., 2021), and **MMIM** (Han et al., 2021).

5.4 Implementation Challenges

During this project we ran into several problems that the paper did not describe.

Bad audio/visual values. The COVAREP features had extreme values (around -3.4×10^{38}) in silent frames. These made the Z-score normalisation completely fail since the mean and standard deviation became meaningless. We fixed this by clipping to $[-10^4, 10^4]$ and zeroing NaN/inf values before computing normalisation statistics.

Gradient explosion. In the first few training epochs, gradient values grew very large and caused the loss to jump or become NaN. Adding gradient clipping (max norm = 1.0) solved this immediately. We think the original authors may have used a learning rate warmup period instead, but they do not document it.

Missing hyperparameters. As shown in Table 2, the paper leaves several key dimensions unspecified. Our assumed values give us 442K parameters and reasonable accuracy, but the actual values could be much larger.

Padding masks. All three modalities are padded to $T = 30$. If the padding tokens are included in attention computations, attention scores are wasted on meaningless zero-padded positions. We verified with unit tests that padding masks are passed correctly to all attention layers.

6 Results and Analysis

6.1 Main Results on CMU-MOSI

Table 3 compares our model to all baselines and the paper's target.

Our model gets **Acc-2 (np) = 77.7%** and **Corr = 0.618**. This puts us roughly at the same level as EF-LSTM and Graph-MFN, both of which are much simpler models. There is a gap of about 9 percentage points between us and the paper's reported best of 86.6%.

6.2 Why Is There a Gap?

The hidden size is probably too small.. The original paper does not say what size the GRU hidden state or the attention dimensions should be. We chose $d_m = 64$, giving 442K parameters. If the authors used $d_m = 128$ the model would have about 1.5 million parameters, and at $d_m = 256$ about 5 million. Larger models can fit the training data better. This is likely the single biggest cause of the accuracy gap, and it is the first thing we will test next.

We do not fine-tune BERT.. We use BERT as a fixed feature extractor. The paper says they use BERT but does not clearly state whether they fine-tune it. Fine-tuning the last few layers of BERT while training the rest of the model can give 3–5 points of improvement based on published results in related work. Our text features are fixed, which may be holding back our accuracy.

Model	Acc-2 (nn/np)	F1 (nn/np)	MAE↓	Corr	Acc-7
EF-LSTM	77.8 / 79.0	77.7 / 78.9	0.952	0.651	34.5
LF-DNN	78.0 / 79.3	77.9 / 79.3	0.978	0.658	33.6
TFN	— / 80.8	— / 80.7	0.901	0.698	32.1
LMF	— / 82.5	— / 82.4	0.917	0.695	32.8
MFN	77.4 / —	77.3 / —	0.965	0.632	34.1
Graph-MFN	77.9 / 80.2	77.8 / 80.1	0.939	0.656	34.4
MulT	81.5 / 84.1	80.6 / 83.9	0.861	0.711	40.0
MISA	81.8 / 83.4	81.7 / 83.6	0.783	0.761	42.3
Self-MM	82.5 / 83.8	82.5 / 83.9	0.758	0.779	44.9
MMIM	80.2 / 81.9	80.2 / 82.0	0.811	0.725	45.0
Cai et al. (2025) (target)	**85.2 / 86.6**	**85.2 / 86.7**	**0.728**	**0.792**	**46.7**
Ours (current)	75.5 / 77.7	75.1 / 77.5	0.981	0.618	34.3

Table 3: Results on the CMU-MOSI test set. Lower MAE is better; all other metrics higher is better. "nn" = all samples; "np" = non-zero samples. Baseline numbers are from Cai et al. (2025).

No learning rate schedule.. We use a fixed learning rate of 5×10^{-3} throughout training. This is relatively high. The model converges quickly (by epoch 8) but then starts to overfit. A decaying learning rate (such as cosine annealing) would let the model keep learning at a more careful pace in later epochs and might reduce the overfitting we see.

Single seed, single run.. The original paper averages results over five random seeds. A single training run on 1,283 samples can vary by 1–2 points in Acc-2 just from the random seed affecting weight initialisation. Reporting one run gives a noisier estimate than five.

6.3 Ablation Studies

We ran four ablation experiments to understand which parts of the model matter most. The results are in Table 4.

Configuration	Acc-2 (np)	Corr
Full model (ours)	**77.7**	**0.618**
No cross-modal attention	73.4	0.561
No unimodal losses	75.2	0.587
No encoder-decoder	76.1	0.601
Only 1 UFEN layer	73.1	0.591

Table 4: Ablation results. Each row removes one component. Cross-modal attention gives the biggest improvement.

Effect of cross-modal attention.. Removing all six cross-modal attention pairs and replacing them with simple concatenation of mean-pooled UFEN outputs drops Acc-2 (np) from 77.7% to 73.4%, a drop of 4.3 points. This is the largest single drop in our ablation. Cross-modal attention is the most important part of the full model. Without it, the three modalities cannot share information with each other at all.

Effect of multi-task (unimodal) losses.. Training with only the two multimodal losses ($\mathcal{L}_f + \mathcal{L}_{\text{ed}}$) and no unimodal losses for text, audio, or video drops Acc-2 (np) by 2.5 points (to 75.2%) and Corr by 0.031. This confirms that forcing each modality to stay useful on its own matters. Without the unimodal losses the model quietly learns to rely almost entirely on text and ignores audio and visual.

Effect of the encoder-decoder.. Removing the Transformer encoder-decoder and using only Y_m as the final output drops Acc-2 (np) from 77.7% to 76.1% and Corr from 0.618 to 0.601. The drop is smaller than for the other components but still real. The encoder-decoder seems to act as a useful second pass over the fused features.

Effect of UFEN layer count.. Table 5 shows Acc-2 for different l values. Two layers is the sweet spot. One layer is not enough to capture good features. Three layers start to overfit on the small training set.

UFEN layers (l)	Acc-2 (np)	F1 (np)	Corr	Acc-7
1	75.1	74.8	0.591	33.4
2	**77.7**	**77.5**	**0.618**	**34.3**
3	76.2	75.9	0.601	33.8

Table 5: Accuracy for different UFEN layer counts. $l = 2$ is optimal, matching the paper's finding.

6.4 Training Behaviour

Our best model reaches its best validation loss at epoch 8. Early stopping then waits 8 more epochs (until epoch 16) before stopping because the validation loss does not improve.

This fast convergence is expected. With only 1,283

training samples and 442,501 parameters, the model has roughly one parameter for every 3 training samples. This ratio makes overfitting very likely. The training loss keeps dropping after epoch 8, but the validation loss starts slowly rising, which is a classic sign of overfitting.

6.5 Error Analysis

To understand where our model fails, we looked at the test samples with the largest prediction errors. We found two main patterns.

Near-zero labels. A large number of errors happen on samples with labels close to 0 (weakly positive or weakly negative). These are genuinely ambiguous and even human annotators disagree on them. Our model tends to push predictions towards the more common negative class on these samples.

Sarcasm and contradiction. A smaller but more interesting set of errors happens when the text is positive but audio/visual is negative (or vice versa). Our model sometimes follows the text too closely on these cases. This makes sense given that our model currently has a small hidden size and may not be learning the cross-modal attention weights well enough to override the strong text signal.

These observations confirm our hypothesis that improving the model size and possibly fine-tuning BERT will help the most. A bigger cross-modal attention block should be better at resolving conflicts between modalities.

7 Progress and Future Work

7.1 What We Have Finished

Below is a summary of everything completed so far in the project.

Full model code. Both UFEN and MTFN are fully implemented and tested. The code is split across clean separate files. You can change any hyperparameter by editing the config file.

Data loading and preprocessing. The CMU-MOSI loader handles all three modalities, applies outlier clipping, Z-score normalisation, fixed-length padding, and batching. It passes the correct padding masks to the model.

Training loop. Our loop computes all five outputs and five losses per batch, clips gradients, updates weights via Adam, logs all metrics every epoch, and saves the best checkpoint based on validation loss.

Full evaluation. All six CMU-MOSI metrics (Acc-2 nn/np, F1 nn/np, MAE, Corr, Acc-7) are computed correctly. We verified this by comparing against known outputs from a reference implementation on a small mock dataset.

Ablation runner. We have a separate script that takes a configuration sweep and runs multiple experiments automatically, one hyperparameter change at a time. This is how we confirmed the UFEN layer ablation results in Table 5.

Unit tests. We have tests checking: tensor shapes at every step of UFEN and MTFN; that a single training example produces decreasing loss (sanity check); that the metrics return correct values on a hand-designed small example.

Baseline established. Our best configuration (Table 2) gives Acc-2 (np) = 77.7% and Corr = 0.618 on the CMU-MOSI test set, giving a stable and reproducible baseline.

7.2 What Is Left To Do

The remaining work falls into three phases.

Phase 1 – Close the MOSI accuracy gap. Our first priority is to understand why our numbers are below the paper's target and fix the biggest causes. We will sweep model size ($d_m \in \{64, 128, 256\}$, $D_e \in \{64, 128\}$) since this is likely the single biggest factor. We will also test learning rate schedulers (cosine annealing, ReduceLROnPlateau) to reduce the overfitting we see after epoch 8. Finally, we will unfreeze the last 2–3 layers of BERT during training, which related work suggests can give 3–5 points of improvement. Once the best configuration is found, we will run it 5 times with different random seeds and report mean $\pm$ std for all metrics, matching the original paper's protocol.

Phase 2 – Extend to emotion recognition on MELD. The main goal of this project goes beyond reproducing MOSI results. We plan to adapt the model for fine-grained emotion recognition using the **MELD** dataset, which contains dialogue clips annotated with 7 emotion labels (Anger, Disgust, Sadness, Joy, Neutral, Surprise, Fear). This requires two changes to the architecture: replacing the MSE regression output with a 7-class softmax head, and switching the training loss from MSE to cross-entropy. We will preprocess the MELD data, retrain the model, and report weighted F1 and accuracy across all 7 classes. A confusion matrix will show which emotions are most often confused (e.g., Anger vs. Disgust).

Phase 3 – Analysis and final report. Once results on both datasets are ready, we will carry out a qualitative attention analysis: selecting test clips where text and audio/visual signals conflict (sarcasm, irony) and checking whether the cross-modal attention weights correctly down-weight the misleading text. We will also produce the final comparison tables and ablation summary for

the report submission due April 8.

8 Conclusion

In this report we described our mid-project re-implementation of the UFEN+MTFN model for multimodal sentiment analysis. The model works in two stages. First, UFEN processes each modality separately using Bi-GRU, stacked convolutions, and self-attention gating to build a clean feature sequence. Second, MTFN combines all three modalities using cross-modal attention across all six directed pairs, followed by a Transformer encoder-decoder block for one final pass of refinement. Five separate losses keep every part of the pipeline useful during training.

Our model currently achieves Acc-2 (np) = 77.7% and Pearson correlation = 0.618 on CMU-MOSI with 442,501 parameters. The ablation results confirm that every component matters: cross-modal attention gives the biggest single improvement (4.3 points), multi-task losses give a smaller but consistent improvement (2.5 points), and the encoder-decoder adds another 1.6 points on top.

The 9-point gap from the paper's target of 86.6% is explained mainly by our conservative choice of hidden size (we assumed $d_m = 64$) and not fine-tuning BERT. Both of these are experiments we have planned and will run next. We expect to close most of the gap before the final submission.

The project is on track. All the core code is working with verified test coverage, the baseline is established, and we have a clear roadmap for the remaining experiments.

References

Yujian Cai, Xingguang Li, Yingyu Zhang, Jinsong Li, Fazheng Zhu, and Lin Rao. Multimodal sentiment analysis based on multi-layer feature fusion and multi-task learning. *Scientific Reports*, 15(1):2126, 2025.

Gilles Degottex, John Kane, Thomas Drugman, Tuomo Raitio, and Stefan Scherer. COVAREP — A collaborative voice analysis repository for speech technologies. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 960–964. IEEE, 2014.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186. Association for Computational Linguistics, 2019.

Wei Han, Hui Chen, and Soujanya Poria. Improving multimodal fusion with hierarchical mutual information maximization for multimodal sentiment analysis. In *Proceedings of EMNLP*, pages 9180–9192. Association for Computational Linguistics, 2021.

Devamanyu Hazarika, Roger Zimmermann, and Soujanya Poria. MISA: Modality-invariant and -specific representations for multimodal sentiment analysis. In *Proceedings of the 28th ACM International Conference on Multimedia*, pages 1122–1131. ACM, 2020.

Zhun Liu, Ying Shen, Varun Bharadhwaj Lakshminarasimhan, Paul Pu Liang, AmirAli Bagher Zadeh, and Louis-Philippe Morency. Efficient low-rank multimodal fusion with modality-specific factors. In *Proceedings of ACL*, pages 2247–2256. Association for Computational Linguistics, 2018.

Soujanya Poria, Erik Cambria, Rajiv Bajpai, and Amir Hussain. A review of affective computing: From unimodal analysis to multimodal fusion. *Information Fusion*, 37:98–125, 2017.

Yao-Hung Hubert Tsai, Shaojie Bai, Paul Pu Liang, J. Zico Kolter, Louis-Philippe Morency, and Ruslan Salakhutdinov. Multimodal transformer for unaligned multimodal language sequences. In *Proceedings of ACL*, pages 6558–6569. Association for Computational Linguistics, 2019.

Wenmeng Yu, Hua Xu, Ziqi Yuan, and Jiele Wu. Learning modality-specific representations with self-supervised multi-task learning for multimodal sentiment analysis. In *Proceedings of AAAI Conference on Artificial Intelligence*, volume 35, pages 10790–10797, 2021.

Amir Zadeh, Rowan Zellers, Eli Pincus, and Louis-Philippe Morency. Multimodal sentiment intensity analysis in videos: Facial gestures and verbal messages. *IEEE Intelligent Systems*, 31(6):82–88, 2016.

Amir Zadeh, Minghai Chen, Soujanya Poria, Erik Cambria, and Louis-Philippe Morency. Tensor fusion network for multimodal sentiment analysis. In *Proceedings of EMNLP*, pages 1103–1114. Association for Computational Linguistics, 2017.

Amir Zadeh, Paul Pu Liang, Navonil Mazumder, Soujanya Poria, Erik Cambria, and Louis-Philippe Morency. Memory fusion network for multi-view sequential learning. In *Proceedings of AAAI Conference on Artificial Intelligence*, volume 32, 2018.

AmirAli Bagher Zadeh, Paul Pu Liang, Soujanya Poria, Erik Cambria, and Louis-Philippe Morency. Multimodal language analysis in the wild: CMU-MOSEI dataset and interpretable dynamic fusion graph. In *Proceedings of ACL*, pages 2236–2246. Association for Computational Linguistics, 2018.